

Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 2

Тема: Скалярні типи, побітові операції та математичне моделювання.

Мета роботи: Дослідити скалярні типи Python, функції math і бітові маски.

Студента III курсу KI-24-1 групи

Ступінь вищої освіти бакалавр
(бакалавр, магістр)

Спеціальність 123 – «Комп'ютерна інженерія»

Освітня програма Програмування на Python. Базовий рівень

Огоновського Олександра Євгенійовича

м. Кременчук 2026 рік

1 Мета роботи та короткі теоретичні відомості

Метою роботи є застосування скалярних типів `int`, `float`, `complex` і `bool` у програмі Python, використання функцій модулів `math` і `cmath`, а також набуття навичок побітового упакування даних у 16-бітний регістр.

Тип `int` у CPython підтримує довгу арифметику: число зберігається масивом цифрових блоків, тому не має фіксованої межі переповнення. Тип `float` реалізує подвійний формат IEEE 754 і містить знак, порядок та мантису; тому порівняння результатів обчислень доцільно виконувати через `math.isclose()`.

Побітове АБО встановлює потрібний біт, побітове І разом з інверсією маски скидає його, а виключне АБО інвертує. Для виділення m -бітного поля, зміщеного на k позицій, використовується вираз $(R \gg k) \& (2^{**m} - 1)$.

2 Постановка індивідуального завдання

Варіант 15. Реалізувати обчислення функції $f(x, y) = (x^2 \cdot \sqrt{y + 1} + \exp(-x)) / \log_2(x^2 + y^2 + 2)$ та перевірити точність результату функцією `math.isclose()`.

Розробити підсистему керування енергоспоживанням із 16-бітним регістром: біт 15 — `SLEEP_MODE`; біти 14..9 — `WAKEUP_SOURCE`; біти 8..3 — `BATTERY_LVL`; біти 2..0 — `SYS_ST`. Необхідно упакувати поля, розпакувати рівень заряду й виконати операції встановлення, скидання та інверсії біта `SLEEP_MODE`.

3 Технологія виконання та програмний код

У каталозі лабораторної роботи створено модулі `scalar_modeling.py` і `register_bitmask.py`, файл конфігурації налагодження `.vscode/launch.json`, правила `.gitignore` та `README.md`. Для математичної моделі використано модуль `math`; для регістра визначено позиції полів, маски та функції упакування і розпакування.

Лістинг файлу `scalar_modeling.py`

```
"""Лабораторна робота № 2: математична модель варіанта 15."""

import math

def compute_power_model(x: float, y: float) -> float:
    """Обчислює f(x, y) для варіанта 15 та перевіряє область визначення."""
    if y < -1.0:
        raise ValueError("Для sqrt(y + 1) параметр y має бути не меншим за -1.")

    numerator = x**2 * math.sqrt(y + 1.0) + math.exp(-x)
```

```

denominator = math.log2(x**2 + y**2 + 2.0)
return numerator / denominator

def main() -> None:
    """Демонструє розрахунок моделі та перевірку точності IEEE 754."""
    x_value = 2.5
    y_value = 3.0
    calculated = compute_power_model(x_value, y_value)

    numerator = x_value**2 * math.sqrt(y_value + 1.0) + math.exp(-x_value)
    denominator = math.log2(x_value**2 + y_value**2 + 2.0)
    reference = numerator / denominator
    is_accurate = math.isclose(calculated, reference, rel_tol=1e-12)

    print("=" * 80)
    print(f{'МАТЕМАТИЧНЕ МОДЕЛЮВАННЯ ВАРІАНТУ 15':^80})
    print("=" * 80)
    print(f"f(x, y) = (x² · √(y + 1) + exp(-x)) / log₂(x² + y² + 2)")
    print(f"Вхідні дані: x = {x_value:.3f}, y = {y_value:.3f}")
    print(f"Чисельник: {numerator:.10f}")
    print(f"Знаменник: {denominator:.10f}")
    print(f"Обчислене f(x, y): {calculated:.10f}")
    print(f"Еталонне значення: {reference:.10f}")
    print(f"Перевірка math.isclose: {is_accurate} (rel_tol=1e-12)")
    print("=" * 80)

if __name__ == "__main__":
    main()

```

Лістинг файлу register_bitmask.py

```

"""Лабораторна робота № 2: 16-бітний регістр варіанта 15."""

SLEEP_MODE_SHIFT = 15
SLEEP_MODE_MASK = 0x1
WAKEUP_SOURCE_SHIFT = 9
WAKEUP_SOURCE_MASK = 0x3F
BATTERY_LEVEL_SHIFT = 3
BATTERY_LEVEL_MASK = 0x3F
SYSTEM_STATE_SHIFT = 0
SYSTEM_STATE_MASK = 0x7
REGISTER_MASK = 0xFFFF

def pack_power_register(sleep_mode: bool, wakeup_source: int, battery_level: int, system_state:
int) -> int:
    """Упаковує параметри керування живленням у 16-бітне машинне слово."""
    if not 0 <= wakeup_source <= WAKEUP_SOURCE_MASK:
        raise ValueError("WAKEUP_SOURCE має належати діапазону 0..63.")
    if not 0 <= battery_level <= BATTERY_LEVEL_MASK:
        raise ValueError("BATTERY_LVL має належати діапазону 0..63.")
    if not 0 <= system_state <= SYSTEM_STATE_MASK:
        raise ValueError("SYS_ST має належати діапазону 0..7.")

    return (
        (int(sleep_mode) << SLEEP_MODE_SHIFT)
        | (wakeup_source << WAKEUP_SOURCE_SHIFT)
        | (battery_level << BATTERY_LEVEL_SHIFT)
        | (system_state << SYSTEM_STATE_SHIFT)
    )

def unpack_power_register(register_value: int) -> dict[str, int | bool]:
    """Виділяє поля регістра зсувами праворуч і масками побітового І."""
    register_value &= REGISTER_MASK
    return {
        "sleep_mode": bool((register_value >> SLEEP_MODE_SHIFT) & SLEEP_MODE_MASK),

```

```

        "wakeup_source": (register_value >> WAKEUP_SOURCE_SHIFT) & WAKEUP_SOURCE_MASK,
        "battery_level": (register_value >> BATTERY_LEVEL_SHIFT) & BATTERY_LEVEL_MASK,
        "system_state": (register_value >> SYSTEM_STATE_SHIFT) & SYSTEM_STATE_MASK,
    }

def set_sleep_mode(register_value: int) -> int:
    """Встановлює біт SLEEP_MODE операцією побітового АБО."""
    return (register_value | (1 << SLEEP_MODE_SHIFT)) & REGISTER_MASK

def clear_sleep_mode(register_value: int) -> int:
    """Скидає біт SLEEP_MODE операціями побітового І та НЕ."""
    return register_value & ~(1 << SLEEP_MODE_SHIFT) & REGISTER_MASK

def toggle_sleep_mode(register_value: int) -> int:
    """Інвертує біт SLEEP_MODE операцією виключного АБО."""
    return (register_value ^ (1 << SLEEP_MODE_SHIFT)) & REGISTER_MASK

def main() -> None:
    """Демонструє упакування, розпакування та модифікацію регістра."""
    sleep_mode = False
    wakeup_source = 17
    battery_level = 45
    system_state = 5
    register_value = pack_power_register(sleep_mode, wakeup_source, battery_level, system_state)
    unpacked = unpack_power_register(register_value)

    print("=" * 80)
    print(f"{'РЕЄСТР КЕРУВАННЯ ЕНЕРГОСПОЖИВАННЯМ ВАРІАНТУ 15':^80}")
    print("=" * 80)
    print(f"Початкові поля: SLEEP={sleep_mode}, WAKEUP={wakeup_source}, BATTERY={battery_level}, SYS_ST={system_state}")
    print(f"DEC: {register_value:d}")
    print(f"HEX: 0x{register_value:04X}")
    print(f"BIN: 0b{register_value:016b}")
    print(f"OCT: 0o{register_value:06o}")
    print("-" * 80)
    print("Розпаковані поля:")
    print(f"SLEEP_MODE      = {unpacked['sleep_mode']}")
    print(f"WAKEUP_SOURCE    = {unpacked['wakeup_source']}")
    print(f"BATTERY_LVL      = {unpacked['battery_level']}")
    print(f"SYS_ST           = {unpacked['system_state']}")
    print("-" * 80)
    after_set = set_sleep_mode(register_value)
    after_clear = clear_sleep_mode(after_set)
    after_toggle = toggle_sleep_mode(after_clear)
    print(f"SET SLEEP_MODE:    0x{after_set:04X} | {after_set:016b}")
    print(f"CLEAR SLEEP_MODE:  0x{after_clear:04X} | {after_clear:016b}")
    print(f"TOGGLE SLEEP_MODE: 0x{after_toggle:04X} | {after_toggle:016b}")
    print("=" * 80)

if __name__ == "__main__":
    main()

```

4 Результати обчислень

Для $x = 2,5$ і $y = 3,0$ значення чисельника дорівнює 12,5820849986, знаменника — 4,1085244568, а значення функції — 3,0624340030. Перевірка `math.isclose()` з відносною точністю 10^{-12} повернула `True`.

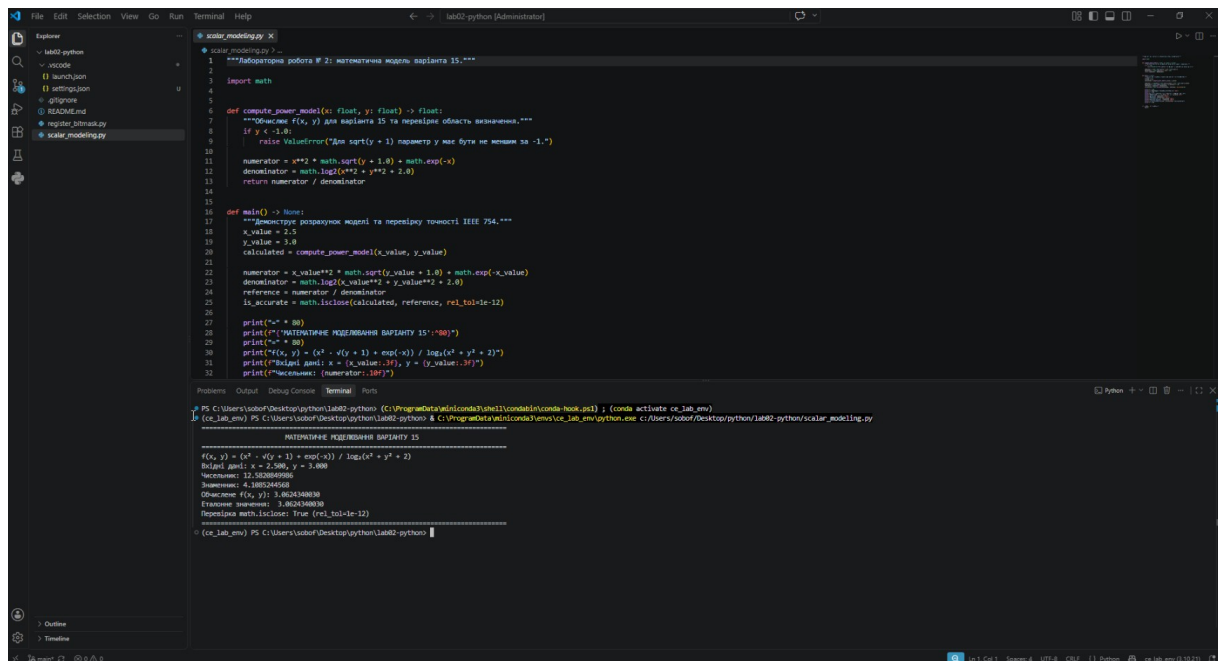


Рисунок 1 – Запуск scalar_modeling.py у VS Code з активним середовищем ce_lab_env

Після упакування полів SLEEP=False, WAKEUP=17, BATTERY=45 і SYS_ST=5 одержано регістр 0x236D. Розпакування повернуло ті самі значення полів, що підтверджує правильність масок і зсувів.

Таблиця 1 – Представлення 16-бітного регістра варіанта 15

Формат	Значення	SLEEP	WAKEUP	BATTERY / SYS_ST
BIN	0b0010001 101101101	0	010001	101101 / 101
OCT	0o021555	—	—	—
DEC	9069	—	—	—
HEX	0x236D	—	—	—

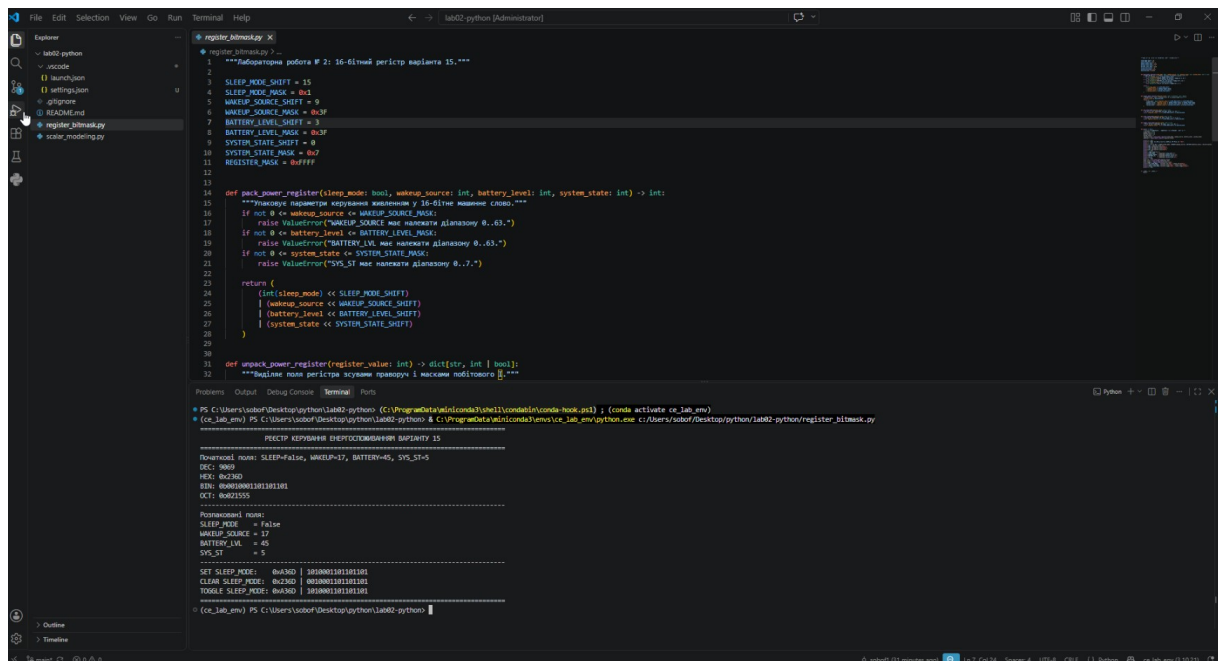


Рисунок 2 – Запуск register_bitmask.py у VS Code з активним середовищем ce_lab_env



Рисунок 3 – Фіксація коміту 8bc2b1f у локальному Git-репозиторії

Віддалений репозиторій GitHub: <https://github.com/sobof1/lab02-python>.

Зафіксований коміт: 8bc2b1f — «feat: виконання лабораторної роботи 2, варіант 15».

Операція SET змінила лише біт 15 і сформувала 0xA36D. Операція CLEAR повернула 0x236D, а TOGGLE знову встановила біт сну. Молодші 15 бітів у кожній операції залишилися незмінними.

5 Відповіді на контрольні питання

5.1 Довга арифметика int

У CPython ціле число подається знаком і масивом цифрових блоків фіксованої внутрішньої розрядності. Пам'ять для масиву виділяється за потреби, тому int може зростати без межі 32- або 64-бітного машинного слова; практичною межею є доступна пам'ять.

5.2 Формат float і точне порівняння

64-бітний float IEEE 754 містить 1 біт знака, 11 бітів зміщеного порядку та 52 біти дробової частини мантиси. Дріб 0,1 нескінченний у двійковій системі, тому зберігається наближено. `math.isclose(a, b, rel_tol, abs_tol)` порівнює різницю з допустимою відносною або абсолютною похибкою.

5.3 Побітова інверсія

Оператор `~x` виконує інверсію у моделі нескінченного знакового двійкового подання Python, тому $\sim x = -(x + 1)$, а $\sim 0 = -1$. Щоб інвертувати лише 16-бітний регістр, результат треба обмежити маскою: $(\sim 0x236D) \& 0xFFFF$.

5.4 Операції над k тим бітом

Для встановлення біта використовується $R \mid (1 \ll k)$, для скидання — $R \& \sim(1 \ll k)$, для інверсії — $R \wedge (1 \ll k)$. Усі ці вирази змінюють тільки біт, заданий одиничною маскою.

5.5 Відмінність math і cmath

`math.sqrt(-4)` породжує `ValueError`, бо модуль `math` працює з дійсними числами. `cmath.sqrt(-4)` повертає комплексний результат `2j`, оскільки `cmath` підтримує комплексну площину.

5.6 Упакування полів A B C

Якщо A займає біти 15..12, B — біти 11..6, C — біти 5..0, то $R = ((A \& 0xF) \ll 12) \mid ((B \& 0x3F) \ll 6) \mid (C \& 0x3F)$. Зворотне вилучення B: $B = (R \gg 6) \& 0x3F$.

6 Висновки

У роботі реалізовано математичну модель варіанта 15 і перевірено відтворюваність обчислення у межах заданої відносної похибки. Використання float потребує порівняння з допуском, оскільки частина десяткових дробів не має точного двійкового подання.

Побітове упакування дозволило подати чотири параметри енергоспоживання одним 16-бітним словом. Маски та зсуви забезпечили коректне виділення полів, а операції SET, CLEAR і TOGGLE змінили лише цільовий біт SLEEP_MODE. Тип `int` зручно використовувати для таких задач, бо він підтримує бітові операції та не обмежений фіксованою розрядною сіткою.